The Encoder–Decoder Architecture

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. Why sequence-to-sequence is hard . . . 3
4. The architecture . . . 4
    4.1 The encoder . . . 4
    4.2 The decoder . . . 5
5. Training . . . 6
    5.1 The data . . . 6
    5.2 The forward pass and teacher forcing . . . 7
    5.3 The loss . . . 8
    5.4 Backpropagation and the update . . . 9
6. Prediction . . . 9
7. Teacher forcing, measured . . . 11
8. Three improvements . . . 12
    8.1 Embeddings instead of one-hot vectors . . . 13
    8.2 Deep (stacked) LSTMs . . . 13
    8.3 Reversing the input . . . 14
9. The original model . . . 14
10. Summary . . . 17
11. Sources . . . 17
12. Key terms . . . 18

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Types of RNN (many-to-one, one-to-many, many-to-many); LSTM (long short-term memory).
- **Leads to:** Attention mechanism; Transformer; Masked self-attention.

1. Overview

Key point: An encoder–**decoder** (G-564) turns one sequence into another of a different length: one LSTM, the **encoder**, reads the input into a **context vector**, and a second LSTM, the **decoder**, writes the output word by word until it produces an end token.

Translating "nice to meet you" into Hindi gives "aap se mil kar achha laga": a sequence goes in and a sequence of a different length comes out. The history of language models calls such tasks **sequence-to-sequence tasks** (G-1772) and places the encoder–decoder (Sutskever et al. 2014) as the first stage of the road to the transformer. This Note opens the architecture up: what sits inside each block, how the whole thing is trained, how it predicts, and the improvements used in the original paper.

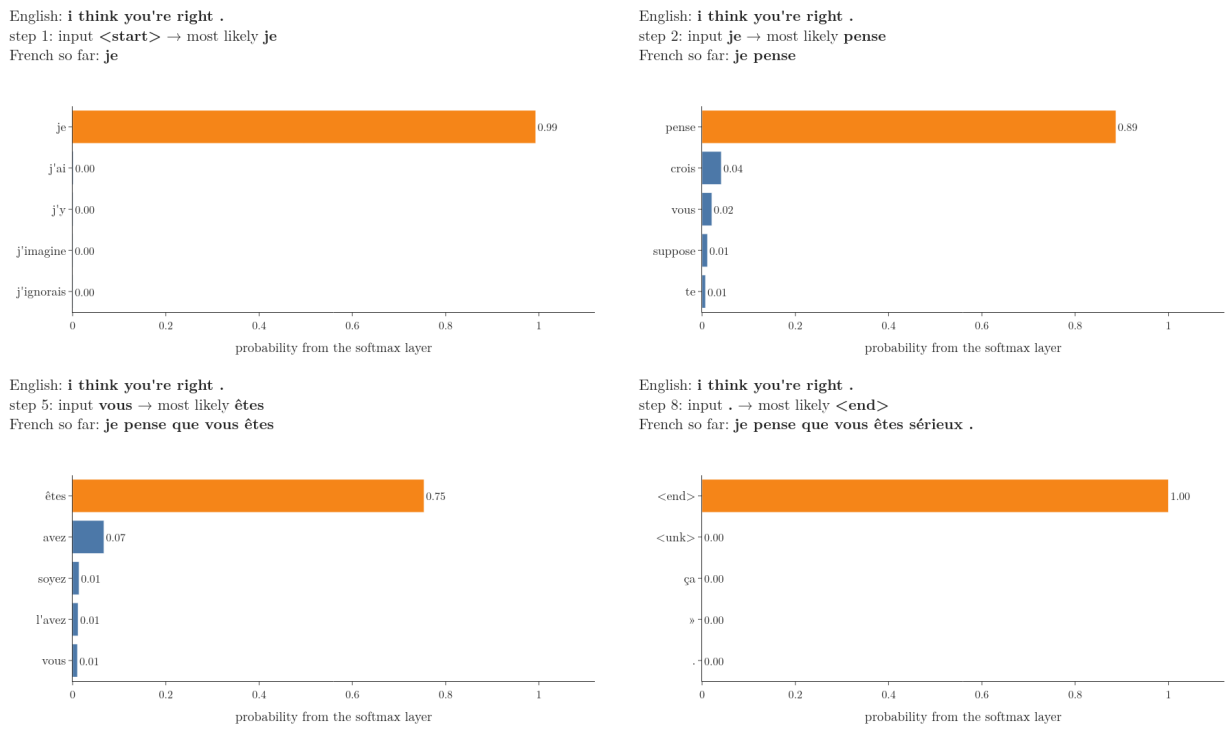


Figure 1: A trained encoder–decoder translating a real English sentence into French, one decoder step per frame: the five most likely words at each step, and the chosen word (orange), which becomes the next input

2. Prerequisites

- The LSTM cell: an RNN with a cell state c_t and a hidden state h_t that reads a sequence one step at a time.
- Many-to-many models of different lengths: many-to-many models whose input and output lengths differ.
- **Word embeddings** (G-2127): word embeddings and Keras' `Embedding` layer.
- Categorical cross-entropy: categorical cross-entropy and **softmax** (G-1830).
- **Backpropagation through time** (G-246): how an unrolled RNN is trained.

3. Why sequence-to-sequence is hard

> **Key point:** Three things vary at once: the length of the input, the length of the output, and the relation between the two lengths.

Machine translation is the running example of this Note. Three difficulties come with it:

1. **The input length varies.** One English sentence has 2 words, another 200.
2. **The output length varies.** The Hindi sentences vary in the same way.
3. **The two lengths are not tied.** "Nice to meet you" has 4 words; "aap se mil kar achha laga" has 6. Nothing guarantees that a 3-word input gives a 3-word output.

Figure 2 shows the third difficulty on the real English–French data of section 6. Figure 2 is a table of counts drawn as coloured cells: each cell holds the number of pairs with that English length (across) and that French length (up), and a darker cell holds more pairs. Read one column: for each English sentence length, the French translations spread over several lengths.

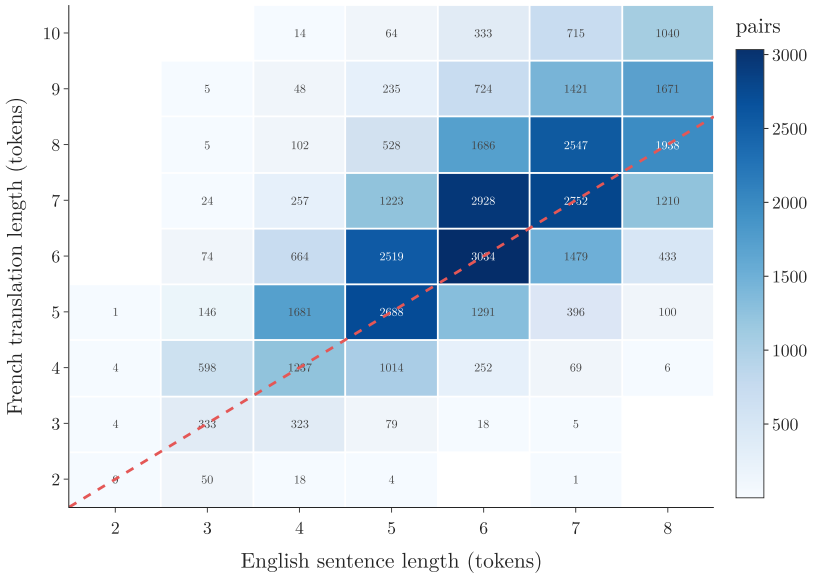


Figure 2: The 40,000 English–French training pairs of section 6 (English up to 8 tokens, French up to 10, punctuation counted), counted by the length of each side. Only 30% of the pairs lie on the dashed diagonal, where both sides have the same length

An LSTM already copes with the first difficulty: it reads one word per step for as many steps as the sentence has. What is new is producing an output whose length the model must decide for itself.

4. The architecture

> **Key point:** Two blocks joined by one vector. The encoder summarises the input into the context vector; the decoder turns the context vector into the output, one word per step.

At the highest level the architecture has three parts:

1. **The encoder** (G-682) receives the input sentence one **token** (G-1981; one word, for now) per step and builds a summary of the whole sentence.
2. **The context vector** is that summary: a fixed list of numbers.
3. **The decoder** (G-564) reads the context vector and writes the output sentence one token per step.

4.1 The encoder

> **Key point:** One LSTM unrolled over the input words. Its hidden and cell states after the last word form the context vector.

The encoder (G-682) is a single LSTM unrolled over time, as in the LSTM cell. At step 1 it reads "think" together with the initial states h_0, c_0 and produces h_1, c_1; at step 2 it reads "about" with h_1, c_1; and so on. Each step updates the states with the new word, so after the last word the pair (h_n, c_n) summarises the whole sentence. This pair is the **context vector** (G-461; Sutskever et al. 2014, section 2; SLP3 §14.7).

The initial states are usually all zeros, which is what Keras uses when no initial state is given (Keras 3.15 `LSTM` docstring). The encoder's outputs at each step are not used; only its final states are.

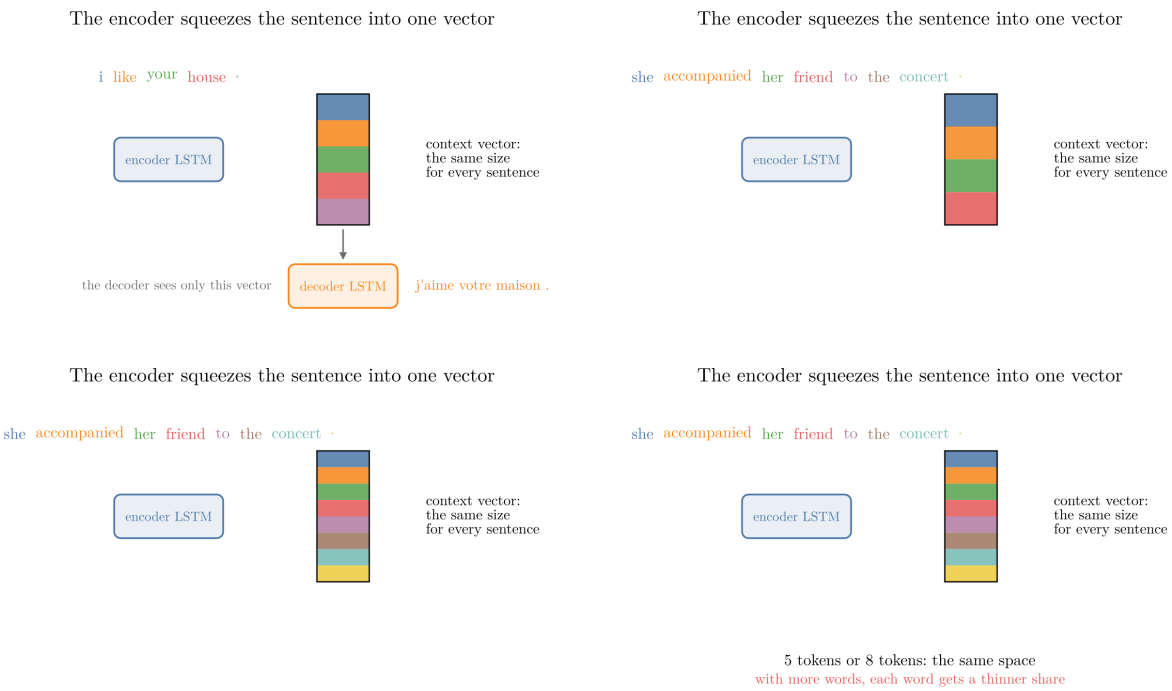


Figure 3: The encoder reads one word per step and ends with one context vector of fixed size, the decoder's only view of the input. A 5-token sentence and an 8-token sentence end in the same space, so each word gets a thinner share. A picture of the idea, not of the LSTM's actual numbers

Figure 3 pictures the consequence. However long the input, **the decoder** (G-564) sees only the fixed-size pair (h_n, c_n); the more words the encoder reads, the less room each word has in it. **Attention** (G-226) measures what this costs on long sentences, in attention against no attention by sentence length.

A GRU (a simpler gated cell, see the four steps of the GRU) or a plain RNN cell can replace the LSTM. Plain RNNs are rarely chosen because of the vanishing gradient problem (see the vanishing gradient through time); the original paper used LSTMs.

4.2 The decoder

> **Key point:** A second LSTM, with its own weights, starts from the context vector, receives a `<start>` token first and then its previous output word, and stops when it outputs `<end>`.

The decoder (G-564) is a different LSTM from the encoder: the two do not share weights. Its initial states are set to the encoder's final states, which is how the decoder learns what the input said. Then:

1. At step 1 it receives a special token, **`<start>`**, which signals "begin writing". The decoder outputs the first word.

2. At step 2 it receives the word from step 1 and the updated states, and outputs the second word.
3. The decoder continues until it outputs the special token **<end>**, and then stops.

The <end> token is what lets the decoder choose the output length.

To output a word, the decoder's hidden state passes through a dense layer with a **softmax** (G-1830) activation, with one unit per word of the output vocabulary. The softmax gives a probability for every word, and the word with the highest probability is the output of that step.

5. Training

> **Key point:** Training uses pairs of sentences. The encoder and **decoder** (G-564) are trained together: forward pass, cross-entropy loss at every output step, backpropagation through both LSTMs, weight update.

5.1 The data

> **Key point:** A parallel corpus: each observation is a sentence and its translation. Words are turned into numbers; the output vocabulary gets two extra tokens, <start> and <end>.

Translation data comes as a **parallel corpus** (G-1442): a table with two columns, a sentence in the source language and its translation in the target language. Each row is one **observation** (G-1374; one training example). The source sentence is the input; the translation is the **target** (G-1949), the output we want the model to produce.

To follow the training by hand, take a corpus of two observations:

English	Hindi
think about it	soch lo
come in	andar aa jao

A network needs numbers, so we first **tokenise** (G-1983; split each sentence into tokens) and then encode each token. The simplest encoding is one-hot (see one column per category):

- English has 5 distinct words: think, about, it, come, in. "think" is $[1, 0, 0, 0, 0]$, "about" is $[0, 1, 0, 0, 0]$.
- Hindi has 5 distinct words: soch, lo, andar, aa, jao. Two special tokens are added, <start> and <end>, for 7 in total. <start> is $[1, 0, 0, 0, 0, 0, 0]$, "soch" is $[0, 1, 0, 0, 0, 0, 0]$.

5.2 The forward pass and teacher forcing

> **Key point:** At every **decoder** (G-564) step the input is the correct previous word from the data, whatever the model predicted. Feeding the gold word is called teacher forcing, and it speeds up training.

Both LSTMs start with random weights. The first observation goes through the network (Figure 4):

1. **The encoder** (G-682) reads "think", "about", "it", one vector per step, and passes its final states to the decoder.
2. The decoder receives `<start>`. The **softmax** (G-1830) gives 7 probabilities; with random weights the largest might belong to "andar", while the correct first word is "soch". The prediction is wrong.
3. The next step needs an input. The plan of section 4.2 says: feed the previous output, "andar". During training we feed the **correct** previous word, "soch", instead.
4. The decoder predicts "jao" where "lo" was correct, and again receives the correct word, "lo".
5. The decoder predicts `<end>`, which is correct, and the sentence is complete.

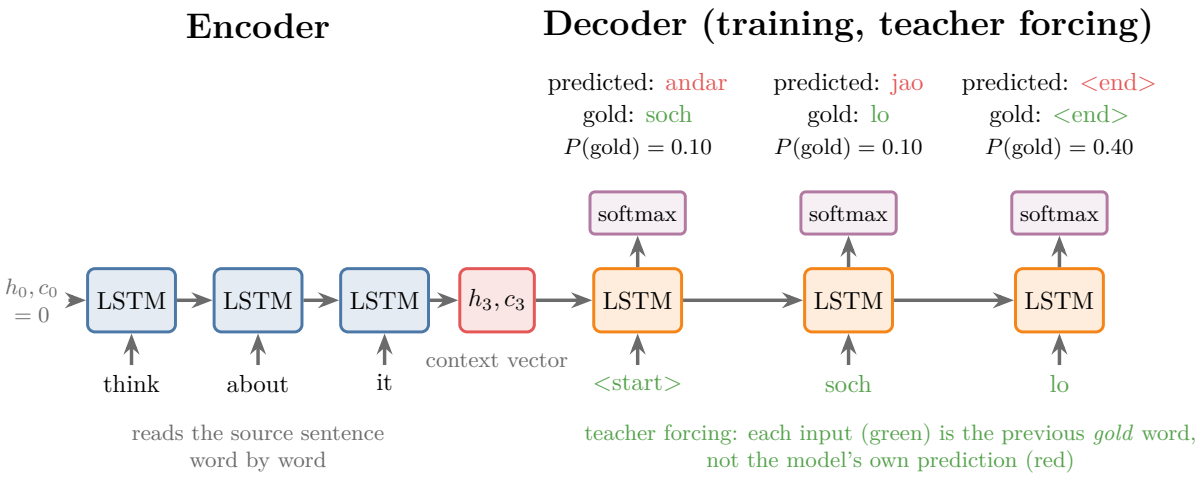


Figure 4: Training on one observation. The decoder's input at each step (green) is the previous gold word, not the model's own prediction (red)

Feeding the gold word instead of the prediction is called **teacher forcing** (G-1955). If a wrong word were fed back, every later step would build on the mistake, and early in training most predictions are wrong. With teacher forcing every step learns from a correct history, which speeds up training (SLP3 §14.7.1). Teacher forcing also fixes the length during training: the decoder runs for exactly as many steps as the gold sentence has tokens, whether or not it predicted `<end>` at the right step. Section 7 measures the effect.

In code, teacher forcing is just a shift. The decoder's input is the gold **target** (G-1949) sentence with `<start>` in front; its target is the same sentence with `<end>` at the back:

Decoder input	`<start>`	soch	lo
Decoder target	**soch**	**lo**	**`<end>`**

5.3 The loss

> **Key point:** Each **decoder** (G-564) step is a classification over the output vocabulary, so the loss is categorical cross-entropy, summed or averaged over the steps.

At every step the decoder picks one word out of 7: a multi-class classification. The loss is **categorical cross-entropy** (G-349; see categorical cross-entropy), computed at every step.

1. **In words:** at each step, take minus the natural log of the probability the model gave to the correct word. Add the steps up (or average them).
2. **Formula:** with V words in the vocabulary and the one-hot target y_t,

$$L_t = -\sum_{k=1}^{V} y_{t,k} \ln \hat{y}_{t,k} = -\ln \hat{y}_{t,\text{gold}}$$

$$L = \sum_t L_t$$

3. **Example:** the **softmax** (G-1830) gave the correct words "soch", "lo" and `<end>` the probabilities 0.1, 0.1 and 0.4.

$$L_1 = -\ln 0.1 = 2.303$$

$$L_2 = -\ln 0.1 = 2.303$$

$$L_3 = -\ln 0.4 = 0.916$$

The total and the mean:

$$L = 2.303 + 2.303 + 0.916 = 5.52$$

$$\text{mean} = 5.52/3 = 1.84$$

Keras' `SparseCategoricalCrossentropy` returns the same mean (Notebook).

The two wrong steps cost much more than the correct one, as they should (Figure 5): $-\ln p$ grows quickly as the probability of the correct word falls.

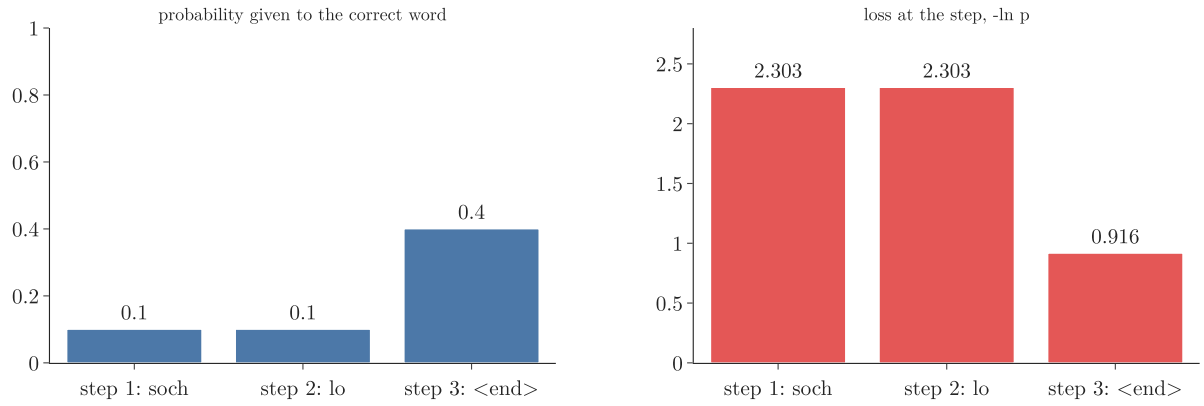


Figure 5: The worked example. Left: the probability the decoder gave each correct word. Right: the loss at each step, $-\ln p$. A probability of 0.1 costs 2.303, one of 0.4 costs 0.916; the mean, 1.84, is what Keras reports

Keras averages over the steps; padding positions are excluded.

5.4 Backpropagation and the update

> **Key point:** The gradient of the loss flows back through the dense layer, **the decoder** (G-564), the context vector and the encoder, so all of them are trained together.

The loss is differentiated with respect to every trainable parameter: the dense layer's weights, the decoder LSTM's weights and **the encoder** (G-682) LSTM's weights. The gradient reaches the encoder through the context vector, the only link between the two blocks. The gradients then update the weights with an optimizer such as SGD or Adam (a rule that turns gradients into weight changes), scaled by the learning rate (see the full training loop of backpropagation through time). The next observation, "come in", goes through the updated network, and so on over the whole corpus for several epochs (an epoch is one full pass over all the training pairs). SLP3 §14.7.1 calls this training **end to end**: the two blocks are never trained separately.

6. Prediction

> **Key point:** With no **target** (G-1949) available, the decoder feeds back its own most likely word at every step (greedy decoding) until it outputs `<end>`. All weights stay fixed.

After training we translate a new sentence:

1. **The encoder** (G-682) reads the sentence and produces the context vector.
2. **The decoder** (G-564) receives `<start>` and outputs the most likely word.
3. That word is fed in as the next input; there is no gold word to force.
4. Steps 2 and 3 repeat until the decoder outputs `<end>`.

Choosing the most likely word at each step is called **greedy decoding** (G-870; SLP3 §14.7). No gradients are computed and no weights change. A mistake at one step is fed into the next one; the toy model of section 5, for instance, could translate "think about it" as "soch jao lo" (Figure 6). Compare Figure 4: there the gold word replaced each mistake; here nothing does.

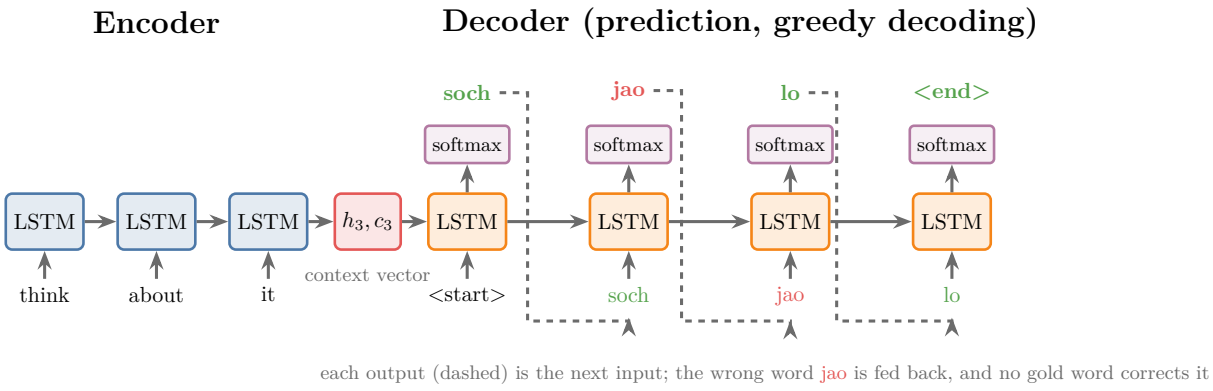


Figure 6: Prediction with greedy decoding on the toy example. Each output becomes the next input (dashed). The wrong word "jao" at step 2 is fed back into step 3, and no gold word corrects it

The Notebook trains a real model of this kind. The data is the English–French corpus of the Keras examples (sentence pairs from the Tatoeba project): 40,000 training pairs with up to 8 English tokens (words and punctuation marks, as in Figure 2), with vocabularies of 6,004 English and 8,004 French tokens (the most frequent words plus the 4 special tokens `<pad>`, `<unk>`, `<start>`, `<end>`; rarer words become `<unk>`). Each block has an embedding of size 128 and an LSTM of 256 units: 4.6 million parameters in all, most of them in the two embeddings and the **softmax** (G-1830) layer. Training for 15 epochs (15 passes over the training pairs) with teacher forcing takes about 5 minutes on a shared GPU. On 1,500 English sentences it never saw, greedy decoding gives, for example:

English (unseen)	Model's French	A reference translation
i like your house .	j'aime votre maison .	j'aime votre maison .
everyone was horrified .	tout le monde a été `<unk>` .	tout le monde a été horrifié .
that doesn't help me .	ça ne me dérange pas .	ça ne m'aide pas .
the pain has mostly gone away .	la douleur a été fabriquée en prison .	la douleur a en majeure partie disparu .

Python: The model in Keras. The decoder's initial states are the encoder's final states; the decoder input is the gold sentence shifted right (**teacher forcing** (G-1955)).

```
ei = keras.Input((None,), dtype="int32")                    # English word
↪  ids
```

```
di = keras.Input((None,), dtype="int32")                    # <start> +
↪  French word ids
x = keras.layers.Embedding(6004, 128, mask_zero=True)(ei)
_, h, c = keras.layers.LSTM(256, return_state=True)(x)      # context
↪  vector (h, c)
y = keras.layers.Embedding(8004, 128, mask_zero=True)(di)
y = keras.layers.LSTM(256, return_sequences=True)(y, initial_state=[h,
↪  c])
out = keras.layers.Dense(8004, activation="softmax")(y)     # one softmax
↪  per step
model = keras.Model([ei, di], out)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

Short, common sentences come out right. Rare words become `<unk>`, because they are outside the vocabulary. Some outputs are fluent French with the wrong meaning: "ça ne me dérange pas" means "that doesn't bother me", and the last example starts correctly ("la douleur a été", "the pain was") and then drifts into nonsense ("made in prison"). Figure 1 shows the decoding of "i think you're right ." step by step. The model picks "je pense que vous êtes" with a clear lead at most steps ("je" 0.99, "pense" 0.89), but at step 6 its top two words are "sérieux" (0.059) and "raison" (0.056, the right word). Greedy decoding takes the top word and never revisits the choice, so the output is "je pense que vous êtes sérieux ." ("I think you are serious").

The **BLEU score** (G-315; a 0 to 100 score for how well the output matches reference translations; see the encoder–decoder stage; SLP3 §13.6.2) over the 1,500 test sentences is 13.1 on a 0–100 scale, counting every French translation in the corpus as a reference. The score is far below the 34.81 of Sutskever et al. (2014), who combined five 384-million-parameter models trained on 12 million sentence pairs (section 9).

7. Teacher forcing, measured

> **Key point:** Trained on its own predictions, the decoder produced no useful translation for the first 3 epochs; with **teacher forcing** (G-1955) it scored from the first epoch. After 10 epochs the teacher-forced model was still slightly ahead (BLEU 10.2 against 9.9).

To see what teacher forcing does, the Notebook writes the same encoder–**decoder** (G-564) as an explicit loop over the decoder steps and trains it two ways on the same 40,000 pairs:

- **teacher forcing:** the input at step t is the gold word $t-1$;
- **own predictions:** the input at step t is the model's own most likely word at step $t-1$, as at prediction time.

Everything else is identical. Each way is trained 3 times with different seeds, and after every epoch the model translates the 1,500 test sentences by greedy decoding.

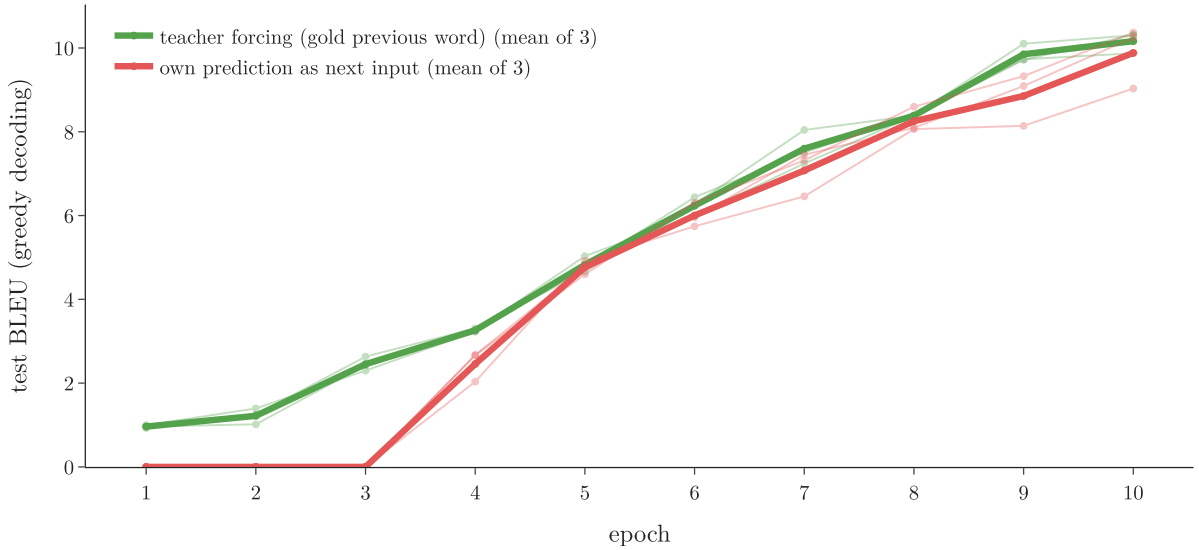


Figure 7: Test BLEU after each epoch. Thin lines: 3 runs each; thick lines: their mean. With teacher forcing (green) the model starts scoring at once; trained on its own predictions (red) it scores 0 for 3 epochs

Mean test BLEU after epoch	1	2	3	4	5	10
Teacher forcing	1.0	1.2	2.5	3.3	4.8	10.2
Own predictions	0.0	0.0	0.0	2.5	4.8	9.9

Early in training almost every prediction is wrong. A decoder fed its own wrong words learns from inputs that have nothing to do with the **target** (G-1949) sentence, and in the first 3 epochs its translations score a BLEU of 0. BLEU takes the geometric mean (a kind of average that is 0 as soon as one of its numbers is 0) of the precisions for word sequences of length 1 to 4 (Papineni et al. 2002, §2.3 and footnote 5), so it is 0 when, for at least one length from 1 to 4 words, not a single word sequence of that length matches a reference. With teacher forcing every step sees the correct history from the first update, so learning starts at once. As the predictions improve, the model's own word more often equals the gold word, the two kinds of input become alike, and the curves meet; on these short sentences the advantage of teacher forcing is concentrated in the first epochs. The result matches SLP3 §14.7.1: teacher forcing "speeds up training".

8. Three improvements

> **Key point:** Real systems replace one-hot vectors with learned embeddings, stack several LSTM layers, and (for some language pairs) feed the source sentence in reverse.

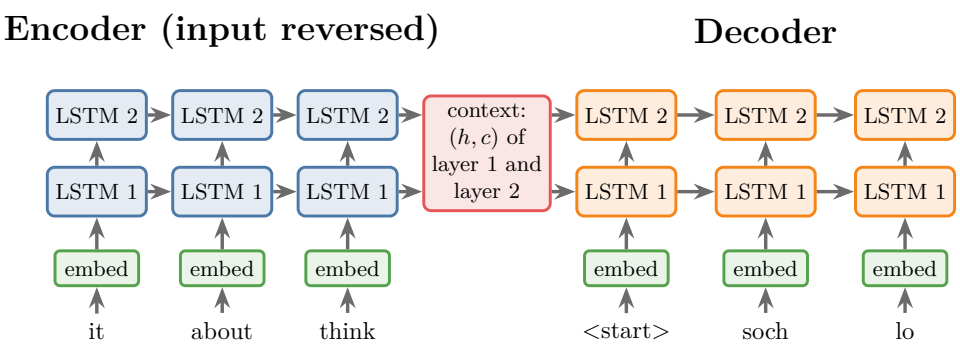


Figure 8: The improved encoder–decoder: embeddings, two stacked LSTM layers in each block, and the source sentence reversed

8.1 Embeddings instead of one-hot vectors

> **Key point:** A one-hot vector is as long as the vocabulary; an embedding is a short, dense, learned vector.

With a vocabulary of 100,000 words, every one-hot input vector has 100,000 numbers, all but one of them 0. An **embedding layer** maps each word to a short dense vector instead, such as 3 or 1,000 numbers, learned so that words used in similar ways get similar vectors (see **word embeddings** (G-2127)). An embedding layer goes in front of the encoder and in front of **the decoder** (G-564). The vectors can come pre-trained (Word2Vec and GloVe are two published sets of ready-made word vectors) or be learned with the model, as in the Notebook, where they fit the translation data. Sutskever et al. (2014) used learned embeddings of 1,000 numbers per word.

8.2 Deep (stacked) LSTMs

> **Key point:** Several LSTM layers on top of each other in each block. The context vector then has one pair of states per layer, which gives the summary more room.

In a **stacked** LSTM (G-1865) the outputs of one layer at each step are the inputs of the layer above, and each layer passes its own states forward in time (Figure 8). **The encoder** (G-682)'s context vector is then the final (h, c) of every layer, and each **decoder** (G-564) layer starts from the matching encoder layer. Three reasons are given for stacking (**deep RNNs** (G-573) treats stacking in full):

1. **More room for the summary.** One layer must squeeze a long sentence into one (h, c) pair; several layers give several. Sutskever et al. (2014, section 3.4) found that deep LSTMs "significantly outperform shallow LSTMs, where each additional layer reduced perplexity by nearly 10%, possibly due to their much larger hidden state". **Perplexity** (G-1492) measures how surprised the model is by the correct words; lower is better.

2. **Levels of abstraction.** Lower layers tend to work closer to the words and higher layers closer to the meaning, as early layers of the visual system detect edges that later layers combine into shapes (SLP3 §14.4.1).
3. **More parameters, more capacity.** More weights let the model capture finer patterns in the data, provided there is enough data to avoid overfitting (learning the training pairs by heart and then failing on new sentences).

The original paper used 4 layers in each block.

8.3 Reversing the input

> **Key point:** Feeding "it about think" instead of "think about it" puts the first source words right next to the first **target** (G-1949) words, which made the original model train much better.

The third trick reverses the order of the source words, but not of the target words. In "think about it" → "soch lo", the word "think" is read first and "soch" is written first, with "about" and "it" in between. Reversed, "think" is the last word the encoder reads, right before the decoder writes "soch". Sutskever et al. (2014, section 3.3) explain the gain this way: the average distance between corresponding words is unchanged, but the first few source words are now very close to the first few target words, so backpropagation "has an easier time establishing communication" between the two sentences. On their English–French task, reversing lowered the test **perplexity** (G-1492) from 5.8 to 4.7 and raised BLEU from 25.9 to 30.6. They also found that LSTMs trained on reversed sentences did much better on long sentences.

9. The original model

> **Key point:** Sutskever et al. (2014) translated English to French with two 4-layer LSTMs of 1,000 units, 1,000-number embeddings and reversed input, and beat a strong phrase-based system.

Figure 9 puts the original model next to the Notebook's model. The original had more than 80 times as many parameters and 300 times as many training pairs:

$$384 \text{ million}/4.6 \text{ million} \approx 83$$

$$12 \text{ million}/40{,}000 = 300$$

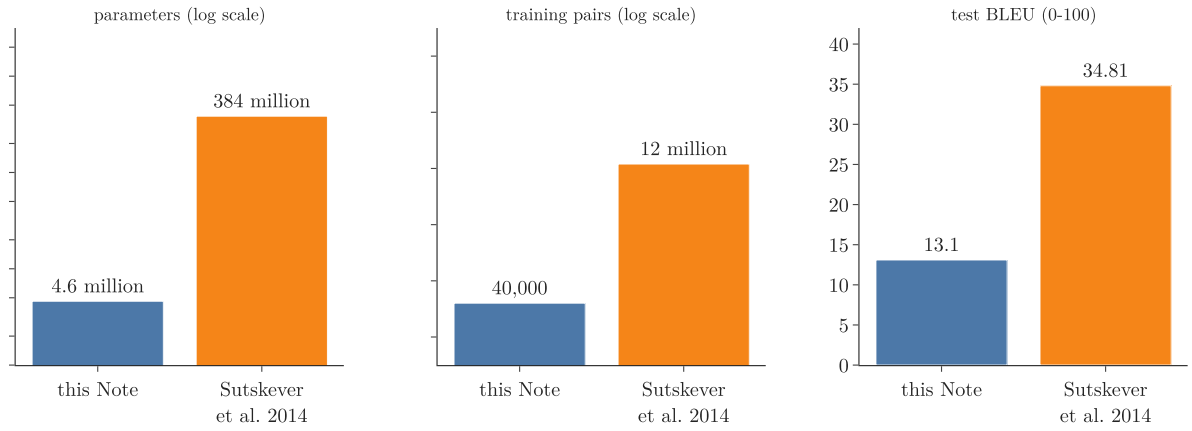


Figure 9: The Notebook's model against the original model of Sutskever et al. (2014): parameters and training pairs (log scales: each tick up multiplies the value, it does not add to it), and test BLEU. The original's BLEU of 34.81 comes from an ensemble of five such models

The details, from Sutskever et al. (2014, sections 3.1–3.6):

- **Task:** English to French translation (WMT'14).
- **Data:** 12 million sentence pairs, with 348 million French words and 304 million English words.
- **Vocabularies:** the 160,000 most frequent English words and the 80,000 most frequent French words; every other word was replaced by an `UNK` token.
- **End token:** one symbol, `<EOS>`, ends every sentence.
- **Input:** source sentences reversed.
- **Embeddings:** 1,000 numbers per word.
- **Network:** 4 LSTM layers in the encoder and 4 in **the decoder** (G-564), with 1,000 units per layer, so the context vector holds 8,000 numbers, one h and one c per layer:

$$h : 4 \times 1{,}000 = 4{,}000$$

$$c : 4 \times 1{,}000 = 4{,}000$$

$$4{,}000 + 4{,}000 = 8{,}000$$

  The model has 384 million parameters.
- **Output:** a **softmax** (G-1830) over the 80,000 French words.

- **Result:** an ensemble of 5 such LSTMs (their predictions combined) reached a BLEU score of 34.81, above the 33.30 of the phrase-based statistical system used as a baseline.

10. Summary

Stage	Encoder	Decoder input at step t	Weights
Training	reads the source sentence	gold word $t-1$ (teacher forcing)	updated by backpropagation
Prediction	reads the source sentence	own predicted word $t-1$	fixed

- The two rows differ because at prediction time there is no gold sentence to feed, so a wrong word at one step is carried into the next.
- **The encoder** (G-682) is an LSTM; its final (h, c) is the context vector, because it is the only link that carries the input to **the decoder** (G-564).
- The decoder is a second LSTM that starts from the context vector, begins with `<start>` and stops at `<end>`, so the decoder chooses the output length itself.
- Each decoder step is a **softmax** (G-1830) classification over the output vocabulary; the loss is cross-entropy over the steps, so a low probability on the correct word costs much more than a high one.
- **Teacher forcing** (G-1955) feeds the gold previous word during training, because a decoder fed its own early wrong words learns nothing at first (BLEU 0 for 3 epochs); prediction feeds the model's own word (greedy decoding).
- Improvements: embeddings, stacked LSTMs, reversed source sentences, because they give shorter input vectors, more room for the summary, and first source words close to the first **target** (G-1949) words.
- The weak point remains the single context vector, because a long sentence must fit in the same fixed size as a short one; **attention** (G-226) removes it.
- This is how one sequence becomes another of a different length: the encoder reads any length, and the `<end>` token lets the decoder write any length.

11. Sources

Built from

- CampusX, "Encoder Decoder | Sequence-to-Sequence Architecture | Deep Learning | CampusX", YouTube, https://www.youtube.com/watch?v=KiL74WsgxoA
- StatQuest with Josh Starmer, "Sequence-to-Sequence (seq2seq) Encoder-Decoder Neural Networks, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=L8HKweZIOmg. 13:00–14:30 (teacher forcing: feed the known correct token, and stop at the known sentence length).

Other references

- Sutskever, I., Vinyals, O. and Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS 2014*. arXiv:1409.3215. Section 2 (the model, `<EOS>`); 3.1 (dataset); 3.3 (reversing the source); 3.4 (training details); Table 1.

- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), web.stanford.edu/~jurafsky/slp3. Chapter 14 (RNNs and LSTMs): §14.4.1 stacked RNNs; §14.7 the encoder–decoder model, greedy choice; §14.7.1 training, teacher forcing. Chapter 13, §13.6.2 (BLEU). Cited as SLP3.
- Papineni, K., Roukos, S., Ward, T. and Zhu, W.-J. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. *Proceedings of the 40th Annual Meeting of the ACL*, Philadelphia, 311–318. Section 2.3 (BLEU: geometric mean of the modified n-gram precisions for n = 1 to 4, times a brevity penalty) and footnote 5 (the geometric average is harsh when any precision vanishes).
- Keras 3.15.1, `keras/src/layers/rnn/lstm.py`: the `LSTM` docstring, call argument `initial_state` ("`None` causes creation of zero-filled initial state tensors"), and `LSTMCell.get_initial_state`, which returns zeros. Also keras.io/api/layers/recurrent_layers/lstm.
- Tatoeba project and manythings.org/anki: the English–French sentence pairs, distributed as `fra-eng.zip` with the Keras examples.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Parallel corpus (G-1442)	A dataset of sentences paired with their translations, one pair per row; it is the training data of a translation model.
Teacher forcing (G-1955)	Feeding the correct (gold) previous token to the decoder during training instead of its own prediction, so every step learns from a correct history and training goes faster.
Greedy decoding (G-870)	Predicting by choosing the most likely token at each step and feeding it back.
Stacked (deep) LSTM (G-1865)	Several LSTM layers on top of each other, each feeding its outputs to the next; the extra layers give the model more room and capacity, with higher layers working closer to the meaning.
`<start>` and `<end>` tokens (G-56)	Special tokens that tell the decoder to begin writing and mark the end of the output.
Decoder (G-564)	The part of a seq2seq model that writes the output sequence.
Sequence-to-sequence (seq2seq) task (G-1772)	A task with a sequence as input and a sequence as output, possibly of different lengths, such as translation.

Term	Meaning
Word embedding (G-2127)	A learned real-valued vector for each word; words used in similar ways get nearby vectors.
Softmax function (G-1830)	A function that turns a list of scores into probabilities that add up to 1: raise e to each score and divide by the total, $e^{z_k} / \sum_j e^{z_j}$; it gives one probability per class in multi-class models.
Backpropagation through time (BPTT) (G-246)	Backpropagation for an RNN: the chain rule runs back over the network unfolded in time, adding up the gradient from every time step at which a shared weight is used.
Encoder (G-682)	The part of a seq2seq model that reads the input sequence and summarises it.
Token (G-1981)	One unit of text the model reads or writes; here, one word.
Context vector (G-461)	The summary of the input that the decoder works from; one fixed vector in the plain encoder–decoder, a new one per output word with attention.
Attention (G-226)	A mechanism that lets each output step weigh all input positions and focus on the useful ones.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Tokenisation (tokenization) (G-1983)	Splitting a text into tokens (words or word pieces), the units a language model reads and writes, before they are turned into vectors.
Categorical cross entropy (G-349)	The loss for classification with more than two classes and a softmax output: the average of minus the log of the probability given to the true class, $-\sum_j y_j \log \hat{y}_j$ per row with one-hot labels; it is small when the true class gets a high probability.
BLEU score (G-315)	A measure of translation quality: how many word sequences of a translation match a human reference.
Deep RNN (stacked RNN) (G-573)	An RNN with two or more recurrent layers stacked on top of each other, all unfolded through time.

Term	Meaning
Perplexity (G-1492)	A score of how surprised a language model is by the correct tokens: the exponential of the mean cross-entropy per token; lower means it gave the correct tokens higher probability.